

Spam Filtering using Naive Bayes Classifier

Joshua Frankie Rayo
2011-19612
CS 280 Programming Assignment 1
September 12, 2016

Contents

1	Introduction	1
2	Methodology	2
2.1	Training	2
2.2	Spam Filter	2
2.3	Testing	3
3	Classifier Improvement	3
3.1	Performance Measures	3
3.2	Laplace Smoothing	3
3.3	Mutual Information	4
4	Program Runtime Speed	6
4.1	Coding the Pythonic Way™	6
4.2	Machine Specifications	7
5	Recommendations	8

1 Introduction

In this paper a spam filter is constructed using a naive Bayes classifier. It has been benchmarked with the TREC public corpora that contains spam and ham emails. Performance measures of this classifier include precision, recall and accuracy. In order to achieve high performance, different approaches have been used: with and without Laplace smoothing, and retaining top words with high mutual information from both spam and ham classes. The classifier is implemented using Python, optimizing code as much as possible (coding in Pythonic Way™), and program speed is also taken into account. Approaches in optimizing Python code will also be discussed relevant to building the spam classifier.

2 Methodology

The classifier is made using Python, a high level, general-purpose programming language that is relatively easy to learn and has very good features in list manipulation.

2.1 Training

For training the classifier, folders 000 to 700 of the trec06p corpus is used. Valid words are taken from each document and its presence or absence is only noted to be added on the spam or ham frequency counts. After counting the number of words, the structure of spam/ham frequency dictionary is as follows:

```
frHam = { 'word1': 12, 'word2': 5, ... }
frSpam = { 'word1': 9, 'word2': 6, ... }
```

To form the vocabulary $|V|$ of unique words one can do the following:

```
alphaDic = list(set(frHam.keys() + frSpam.keys()))
```

After training, the prior probabilities for each class are given by $P(ham) = 7523/21300 = 35.3192\%$ and $P(spam) = 13777/21300 = 64.6808\%$ with dictionary size of 54, 189.

2.2 Spam Filter

The spam classifier is implemented as a class with the following attributes and methods:

- **cHam/cSpam** - count of ham/spam documents in the training set
- **frHam/frSpam** - frequency dictionary of words from the training set
- **fileLabels** - true labels of testing files
- **count** - array containing the number of true/false positives/negatives
- **isLS** - stores the value of λ used in smoothing
- **extract_informative()** - extracts 200 most informative words based on information gain for the whole training set.
- **classify(fileName, wList, mode)** - classifies a file given its wordlist by comparing $P(ham|d)$ and $P(spam|d)$, choosing the right dictionary. Attribute **count** is updated in this method. If $P(ham|d) \leq P(spam|d)$, the document is classified as spam.
- **prob(type, vector, src)** - calculates the posterior probability of a document by forming the feature vector (from the appropriate dictionary) and multiplying the factors, applying Laplace smoothing if necessary.
- **stats()** - calculates the precision, recall and accuracy of the classifier after testing

2.3 Testing

Folders 071 to 126 are used for testing the classifier. Without lambda smoothing, the performance measures are the following:

- Precision: 97.7765%
- Recall: 88.4598%
- Accuracy: 90.8667%

3 Classifier Improvement

3.1 Performance Measures

Evaluating a spam filter usually consists of three metrics: precision, recall and accuracy which are derived from the number of true/false positives/negatives.

- $P = \frac{TP}{TP+FP}$: higher precision means fewer false positives
- $R = \frac{TP}{TP+FN}$: lower recall means more false negatives
- $A = \frac{TP+TN}{TP+TN+FP+FN}$: ratio of correctly classified messages

Different approaches in classifying will also yield different performance. In this paper, lambda smoothing and mutual information are used to improve the classifier.

3.2 Laplace Smoothing

This approach is used to avoid zero probabilities in classifying a document. This is caused by evaluating a word not existing from the training data:

$$P(w_i|ham) = \frac{frHam[w_i] + \lambda}{cHam + \lambda * |dict|}$$

The probability is defined similarly for the spam class.

The table below contains the performance of classifier (in percent) by using different values of λ , rounded up to 4 decimal digits:

λ	precision	recall	accuracy
0.000	97.7765	88.4598	90.8667
0.005	97.7816	88.6709	91.0090
0.010	97.7831	88.7292	91.0493
0.050	97.7837	88.7539	91.0649
0.100	97.7845	88.7903	91.0895
0.500	97.7863	88.8654	91.1401
1.000	97.7885	89.9512	91.1979
2.000	97.7906	89.0379	91.2564

Clearly, $\lambda = 2$ takes the lead in classifier performance using lambda smoothing.

3.3 Mutual Information

Mathematical Model

The spam classifier can be improved by reducing the dictionary only containing informative words. Obtaining these involves computing for the mutual information[2] of a word w :

$$I(w) = \sum_{x \in \{0,1\}} \sum_{c \in C} P(x, c) \log \frac{P(x, c)}{P(x)P(c)}$$

The notation $x \in \{0, 1\}$ means presence or absence of w . This measures how well w discriminates between the various classes[1]. Which means, if $I(w)$ is high, then w is indeed informative for the spam filtering problem. The summation above is further expanded:

$$\begin{aligned} I(w) &= I_h(w) + I_s(w) \\ I_h(w) &= P(w \text{ present}, \text{ham}) \log \frac{P(w \text{ present}, \text{ham})}{P(w \text{ present})P(\text{ham})} + \\ &\quad P(w \text{ absent}, \text{ham}) \log \frac{P(w \text{ absent}, \text{ham})}{P(w \text{ absent})P(\text{ham})} \\ I_s(w) &= P(w \text{ present}, \text{spam}) \log \frac{P(w \text{ present}, \text{spam})}{P(w \text{ present})P(\text{spam})} + \\ &\quad P(w \text{ absent}, \text{spam}) \log \frac{P(w \text{ absent}, \text{spam})}{P(w \text{ absent})P(\text{spam})} \end{aligned}$$

Where $I_h(I_w)$ is information gain of w in ham(spam) class.

Most Informative Words

Using this definition, top 200 words for each class and overall are obtained. The MI values are based on the natural logarithm. **The most informative ham words are:**

['the' -i 1.7683, 'i' -i 1.3834, 'it' -i 1.3616, 'if' -i 1.2211, 'on' -i 1.2169, 'our', 'mime', 'to', 'board', 'list', 'my', 'format', 'that', 'handyboard', 'use', 'thanks', 'but', 'there', 'can', 'would', 'anyone', 'of', 'have', 'so', 'am', 'v', 'hb', 'know', 'apache', 'opt', 'and', 'does', 'company', 'r', 'handy', 'me', 'problem', 'work', 'mail', 'm', 'allowed', 'what', 'how', 'public', 'any', 'ic', 'be', 't', 'version', 'smtp', 'just', 'do', 'university', 'sender', 'code', 'robot', 'some', 'website', 'motor', 'microsoft', 'an', 'l', 'z', 'n', 'is', 'mozilla', 'campaign', 'as', 'ephedra', 'outlook', 'stock', 'which', 'file', 'port', 'info', 'u', 'corp', 'ra', 'not', 'one', 'html', 'fred', 'same', 'other', 'market', 'then', 'message', 'express', 'body', 'discussion',

'get', 'mailing', 'motors', 'foxmail', 'problems', 'should', 'o', 'question', 'two', 'news', 'e', 'p', 'also', 'running', 'pine', 'gmt', 'promotional', 'writely', 'input', 'host', 'output', 'however', 'or', 'computer', 'cantex', 'like', 'sensor', 'ezmlm', 'effects', 'mimeole', 'think', 'gold', 'was', 'help', 'could', 'a', 'tried', 'hoodia', 'oil', 'when', 'discreet', 'system', 'power', 'seems', 'pm', 'them', 'visit', 'shareholder', 'cren', 'beta', 'are', 'produced', 'web', 'product', 'files', 'med', 'someone', 'find', 's', 'control', 'satisfaction', 'hungry', 'time', 'sonar', 'working', 'line', 'approved', 'x', 'able', 'chip', 'fine', 'data', 'in', 'program', 'investors', 'send', 'they', 'potential', 'sensors', 'photoshop', 'watch', 'error', 'bulk', 'test', 'page', 'probably', 'about', 'b', 'voltage', 'source', 'lcd', 'need', 'verdana', 'xp', 'using', 'digital', 'internet', 'you', 'different', 'trying', 'way', 'science', 'mode', 'engineering', 'try', 'connect' -i 1.0636, 'hello' -i 1.0635, 'between' -i 10.0634, 'ir' -i 1.0633, 'eudora' -i 1.0632].

The top 200 spam words are:

[the' -i 0.6134, 'handyboard' -i 0.5380, 'i' -i 0.5353, 'list' -i 0.5335, 'it' -i 0.5212, 'board', 'thanks', 'if', 'hb', 'use', 'my', 'anyone', 'handy', 'on', 'there', 'ic', 'but', 'would', 'am', 'robot', 'does', 'problem', 'motor', 'work', 'that', 'know', 'university', 'so', 'our', 'can', 'port', 'mail', 'code', 'fred', 'to', 'version', 'mime', 'motors', 'me', 'discussion', 'have', 'output', 'file', 'format', 'input', 'sensor', 'what', 'how', 'mozilla', 'running', 'host', 'pine', 'of', 'mailing', 'files', 'seems', 'cren', 'sonar', 'question', 'any', 'problems', 'just', 'same', 'however', 'sensors', 'chip', 'some', 'voltage', 'lcd', 'beta', 'do', 'gmt', 'which', 'and', 'be', 'source', 'probably', 'computer', 'servo', 'an', 'ir', 'connect', 'someone', 'tried', 'circuit', 'pm', 'mode', 'then', 'error', 'other', 'interface', 'page', 'two', 'engineering', 'test', 'analog', 'should', 'connected', 'as', 'smtp', 'system', 'power', 'web', 'fine', 'ports', 'working', 'v', 'think', 'advance', 'simple', 'one', 'display', 'battery', 'control', 'bit', 'line', 'hardware', 'forum', 'seem', 'digital', 'mike', 'able', 'either', 'dmdx', 'bain', 'machine', 'martin', 'not', 'hello', 'appreciated', 'get', 'science', 'r', 'also', 'correct', 'assembly', 'company', 'ascii', 'pin', 'device', 'data', 'john', 'function', 'done', 'point', 'routines', 'eudora', 'hope', 'm', 'type', 'wondering', 'library', 'etc', 'robotics', 'find', 'programming', 'different', 'could', 'send', 'microsoft', 'parts', 'messages', 'dc', 'maybe', 'between', 'public', 'systems', 'like', 'help', 'ok', 'pcode', 'them', 'boards', 'is', 'outputs', 'linux', 'serial', 'servos', 'was', 'couple', 'put', 'enough', 'trying', 'ribbon', 'else', 'polaroid', 'values', 'department', 'default', 'anybody', 'both', 'when', 'signal', 'none', 'helo', 'resistor' -i 0.4472, 'apache' -i 0.4471, 'built' -i 0.44709, 'plan' -i 0.44707, 'lego' -i 0.44704].

The top 200 infomative words in the whole problem are:

[the' -i 2.3818, 'i' -i 1.9187, 'it' -i 1.8829, 'if' -i 1.7280, 'list' -i 1.7263, 'board', 'on', 'handyboard', 'our', 'my', 'mime', 'to', 'thanks', 'use', 'that', 'but', 'there', 'format', 'anyone', 'hb', 'would', 'can', 'handy', 'am', 'so', 'know', 'does', 'have', 'of', 'problem', 'work', 'ic', 'me', 'mail', 'v', 'and', 'robot', 'what', 'university', 'motor', 'apache', 'opt', 'how', 'version', 'company', 'r', 'code', 'any', 'port', 'm', 'be', 'just', 'allowed', 'public', 'mozilla', 'fred', 'do', 'file', 'some', 'smtp', 'discussion', 'motors', 't', 'an', 'which', 'same', 'as', 'microsoft', 'output', 'mailing', 'input', 'sender', 'running', 'pine', 'is', 'sensor', 'website', 'problems',

'host', 'question', 'l', 'n', 'then', 'other', 'one', 'outlook', 'z', 'not', 'campaign', 'however', 'gmt', 'info', 'seems', 'stock', 'should', 'two', 'cren', 'computer', 'get', 'ephedra', 'files', 'u', 'message', 'sonar', 'html', 'beta', 'also', 'corp', 'tried', 'ra', 'express', 'chip', 'think', 'sensors', 'pm', 'body', 'system', 'market', 'someone', 'power', 'voltage', 'lcd', 'like', 'web', 'or', 'could', 'probably', 'o', 'help', 'source', 'was', 'control', 'e', 'news', 'p', 'working', 'error', 'when', 'page', 'foxmail', 'fine', 'test', 'line', 'connect', 'ir', 'them', 'able', 'servo', 'mode', 'find', 'engineering', 'mimeole', 'promotional', 'writely', 'data', 'digital', 'interface', 'circuit', 'connected', 'are', 'send', 'a', 'time', 'simple', 'battery', 'effects', 'science', 'cantex', 'advance', 'hello', 'bit', 'gold', 'ezmlm', 'program', 'analog', 'different', 'seem', 'either', 'eudora', 'hope', 'produced', 'oil', 'trying', 'internet', 'bulk', 'between', 'they', 'done', 'hoodia', 'point', 'discreet', 'ports', 'visit', 'put', 'display', 's' - $\hat{z}$ 1.5106, 'product' - $\hat{z}$ 1.5105, 'mike' - $\hat{z}$ 1.5104, 'need' - $\hat{z}$ 1.5103, 'machine' - $\hat{z}$ 1.5102]

Note that the top words for ham has greater information gain than the top words for spam.

Reducing the Dictionary

In this classification approach, the dictionary only consists of top 200 informative words. Together with lambda smoothing of value $\lambda = 2.0$, the performance measures are the following:

- Precision: 98.3778%
- Recall: 70.2559%
- Accuracy: 79.1732%

The classifier gained higher precision but compromising recall and accuracy.

4 Program Runtime Speed

4.1 Coding the Pythonic WayTM

While the usual syntax exists in Python, one can combine multiple lines of code in one using idiomatic Python statements. This makes the runtime more efficient by using better libraries and existing utility functions.

Regular Expressions

Instead of using `if` and `split` statements to analyze a token, we use regular expressions and let Python do validations. Following the definition of a valid word (purely alphabetic and contains a comma or period at the end), the regular expression is given by `r'([a-zA-Z]+)([.,]?)'`, and the valid word is stored in the frequency dictionary after removing comma or dot at the end.

Faster File Reading

Calling the usual `read()` makes program run slower than using `os.read()` function. The latter function can read a file by specifying a buffer length. For the program to run faster, the author set this to 1MB.

List Manipulation

Using Python enables users to exploit nice features of list manipulation and comprehension techniques[6]. One can declare anonymous functions as a parameter using `lambda`, map list elements from one to another using an anonymous function without using for loops (`map`), filter elements of a list subject to a condition, (`filter`) and combine all elements of a list (`reduce`). Implementing a task using multiple lines are shortened by one line!

Serializing the Classifier

One can store the state of an object via serializing it to a file. In the program, it serializes the classifier's state after training it. Rather than processing the input repeatedly, the program will just load the classifier and proceed to testing right away. The program also serializes the test inputs and its valid words before the testing itself so that parsing will not be repeated anymore.

1. `training.o` already contains the valid words from folders 001 to 070 to bypass parsing.
2. `model.o` already contains the classifier itself and bypass model training.
3. `testing.o` already contains the valid words from folders 001 to 070 to bypass parsing and proceed to testing immediately.

The Decimal Library

For computing the posterior probabilities, the issue of underflow is caused by multiplying very small numbers in many factors. One must use exponential logarithm to avoid underflow! Fear not, `Decimal` library ables the programmer to carry these kinds of calculations avoiding exp-log without suffering from underflow but still maintaining accuracy [7].

4.2 Machine Specifications

The code is run in a machine with the following features:

- Mac Pro (early 2009)
- OS X El Capitan v10.11.6
- 2×2.66 GHz Quad-Core Intel Xeon
- 16 GB 1066 MHz DDR3
- NVIDIA GeForce GT 120 512 MB

Training Time

Parsing the documents and training the classifier only takes 21 seconds using efficient code. Calling the usual libraries make the time take $6\times$ (before the author optimizes his code) which is crippling slow.

Testing Time

Using only 200 informative words takes ~ 104 seconds to classify all testing data. While using the original dictionary take ~ 2076 seconds, for each value of λ . Whole testing take ~ 17079 seconds, and output is written to `outputs.txt` with the following text:

- 0.0 [Decimal('0.9777645423863410760373238039'), Decimal('0.8845981140547822182308037719'), Decimal('0.9086672315700278416656579107')]
- 0.005 [Decimal('0.9778162911611785095320623917'), Decimal('0.8867085765603951504265828469'), Decimal('0.9100895775329863212686115482')]
- 0.01 [Decimal('0.9778305621536025336500395883'), Decimal('0.8872923215087561742254153570'), Decimal('0.9104829923738046241375136182')]
- 0.05 [Decimal('0.9778365943552576248546763302'), Decimal('0.8875392905253704535249214189'), Decimal('0.9106494371141508291974337247')]
- 0.1 [Decimal('0.9778454721683744115203544724'), Decimal('0.8879030085316569375841939829'), Decimal('0.9108945648226606948311342452')]
- 0.5 [Decimal('0.9778637898377666145104175245'), Decimal('0.8886543930549318964226912139'), Decimal('0.9114009603357139974982851148')]
- 1.0 [Decimal('0.9778846560697310334127868436'), Decimal('0.8895118352684585284495477580'), Decimal('0.9119788334169159735071852249')]
- 2.0 [Decimal('0.9779057294684799092556746027'), Decimal('0.8903794342164346654692411316'), Decimal('0.9125635516281321873865149498')]
- MI [Decimal('0.9834066624764299182903834067'), Decimal('0.7025594970812752581948810058'), Decimal('0.7915506597264253722309647742')]

5 Recommendations

To improve this work, future collaborators may do the following:

- If a document has equal probabilities of being spam or ham, the program classifies it as spam. Try to change the relationship if it has greater performance and consequently optimal λ may also change.

- Try to remove most frequent and/or rare words[1] from the dictionary and compare its performance versus two approaches demonstrated in this paper.
- Multithreading in its purest sense is impossible with Python due to its Global Interpreter Lock (GIL) [8]. One can program in multithread but the reduction in runtime speed is not significant. It is recommended to use lower-level languages like C or Fortran and use OpenMPI library to incorporate parallel/multithreaded programming that will greatly improve the classifier's runtime speed.

References

- [1] Johan Hovold. *Naive Bayes spam filtering using word-position-based attributes and length-sensitive classification thresholds*. Department of Computer Science, Lund University, Sweden.
- [2] Mai Vu. *Lecture on entropy and mutual information*. McGill University.
- [3] Le Zhang, Jingbo Zhu, Tianshun Yao. *An Evaluation of Statistical Spam Filtering Techniques*. Institute of Computer Software and Theory, Northeastern University, China.
- [4] PythonSpeed. *Performance Tips* <https://wiki.python.org/moin/PythonSpeed/PerformanceTips>
- [5] StackOverflow. *Optimizing Python Code* <http://stackoverflow.com/questions/7165465/optimizing-python-code>
- [6] Jeff Knupp. *Writing Idiomatic Python* <https://jeffknupp.com/blog/2012/10/04/writing-idiomatic-python/>
- [7] Python 2.x. *Decimal library* <https://docs.python.org/2/library/decimal.html>
- [8] Nick Coghlan. *Efficiently Exploiting Multiple Cores in Python* http://python-notes.curious efficiency.org/en/latest/python3/multicore_python.html